PROJECT REPORT

ON

Quizly: Intelligent MCQ Generation & Assessment System

By

Priyanka Kumari

INDEX

Sr. No.	Contents	Page No.
1	Introduction	**3**
2	Problem Statement	**3-4**
3	Scope of Proposed System	**4-5**
4	Objectives of Proposed System	**5**
5	Methodology	**5-6**
5.1	Flow Diagram	**6**
5.2	Description of Working	**7**
5.3	Functional Requirements	**8**
5.4	Non-functional Requirements	**8-9**
6	Technical Requirements	**9-10**
6.1	Software Requirements	**10**
7	Expected Outcomes	**11**
8	Conclusion	**11-12**

1. Introduction

The rapid growth of Artificial Intelligence (AI) has transformed modern education by enabling personalized and intelligent learning experiences. Traditional online examination systems primarily depend on predefined question banks that require continuous manual updates and may become repetitive after repeated usage. To overcome these limitations, this project presents an AI-Powered Personalized MCQ Generator and Assessment Platform that dynamically generates Multiple-Choice Questions (MCQs) using Large Language Models (LLMs).

The proposed system is developed using React.js for the frontend, FastAPI for the backend, and PostgreSQL for database management. Google Gemini is integrated through the google.generativeai Python SDK to generate customized MCQs based on the user's selected subject, topic, difficulty level, and number of questions.

The platform provides user registration and login, timed examinations, automatic evaluation, instant results, answer explanations, and test history management. Each generated question includes four answer options, a correct answer, an explanation, and a classification according to Bloom's Taxonomy. The system also maintains topic-wise performance records and generates rule-based learning recommendations based on assessment scores. These features provide learners with a personalized environment for academic preparation, technical learning, and placement interview practice.

2. Problem Statement

Working of Existing System

Existing online quiz and assessment platforms primarily rely on predefined question banks. Although these systems support online examinations, they have several limitations:

- Dependence on manually created question banks.
- Repetition of questions after repeated usage.
- Limited customization according to individual learning requirements.
- Considerable manual effort required to create and maintain question databases.
- Limited integration of AI-generated answer explanations.

- Insufficient topic-wise performance tracking in some conventional systems.
- Limited support for personalized learning recommendations.
- Difficulty in generating questions according to different cognitive learning levels.

Need of New System

To address these limitations, there is a need for an intelligent assessment platform that can dynamically generate topic-specific questions using Artificial Intelligence.

The proposed system aims to:

- Generate customized MCQs based on user-selected parameters.
- Reduce manual effort in question preparation.
- Personalize assessments according to subject, topic, and difficulty.
- Provide timed online tests with automatic submission.
- Evaluate responses and display results immediately.
- Provide explanations for correct answers to reinforce learning.
- Maintain test history and topic-wise performance records.
- Generate learning recommendations based on assessment scores.
- Classify questions according to Bloom's Taxonomy.

3. Scope of Proposed System

The proposed system focuses on providing AI-generated online assessments for students, job aspirants, and independent learners.

The system allows users to:

- Register and log in using their credentials.
- Generate customized tests by selecting the subject, topic, difficulty level, and number of questions.
- Specify the test duration.
- Generate MCQs using the Google Gemini API.
- Attempt questions within a specified time limit.

- Submit tests manually or through automatic submission when the timer expires.
- Receive automatically calculated scores and evaluation results.
- View correct answers and AI-generated explanations after submission.
- Access previous test records and scores.
- Track topic-wise performance and accuracy.
- Receive rule-based learning recommendations based on test scores.
- Download assessment reports in the supported HTML format.

The platform can be extended in the future to support educational institutions, corporate training programs, certification examinations, advanced analytics, and additional AI models.

4. Objectives of Proposed System

The main objectives of the proposed system are:

- To develop an AI-powered online assessment platform.
- To generate MCQs dynamically using a Large Language Model.
- To personalize assessments according to user preferences.
- To implement user authentication using JSON Web Tokens (JWT).
- To conduct timed online examinations.
- To automatically evaluate user responses and calculate scores.
- To maintain test history using a relational database.
- To track topic-wise performance and accuracy.
- To provide personalized learning recommendations based on assessment results.
- To classify generated questions according to Bloom's Taxonomy.
- To provide AI-generated explanations for correct answers.
- To provide a user-friendly interface for test creation, examination, and result analysis.

5. Methodology

(a) Flow Diagram Followed by Description of Working

System workflow:

User Registration / Login

↓

React.js Frontend

↓

FastAPI Backend

↓

JWT Authentication and Request Validation

↓

User Selects Subject, Topic, Difficulty, Question Count, and Duration

↓

Backend Creates a Test Session

↓

Google Gemini API Generates MCQs

↓

Backend Parses and Validates the Generated Questions

↓

Correct Answers and Explanations Are Kept in Server-Side Temporary Storage

↓

Questions Are Displayed in the React Interface

↓

User Attempts the Timed Test

↓

Manual Submission or Automatic Submission on Timeout

↓

Backend Evaluates the Answers

↓

Scores and Test Results Are Stored in PostgreSQL

↓

Topic Performance and Learning Recommendations Are Updated

↓

Results and Explanations Are Displayed

↓

User Can Download the Assessment Report

Description of Working

1. The user registers by providing an email address and password.
2. The backend hashes the password and stores the user details in PostgreSQL.
3. During login, the backend verifies the credentials and generates a JWT access token.
4. The frontend uses the token to authenticate subsequent API requests.
5. The user selects the subject, topic, difficulty level, number of questions, and test duration.
6. The FastAPI backend creates a test record and sends a generation request to the Gemini API.
7. Gemini generates MCQs containing a question, four options, the correct answer, an explanation, and a Bloom's Taxonomy level.
8. The backend processes the AI response, parses the JSON, and prepares the questions for the examination interface.
9. The complete question data, including correct answers and explanations, is retained in temporary server-side memory. The correct answers and explanations are excluded from the questions initially returned to the frontend.
10. The user attempts the test through the React interface, which tracks selected answers and the remaining time.
11. When the user submits the test or the timer expires, the backend retrieves the original questions and evaluates the submitted answers.
12. The system calculates the number of correct, incorrect, and unanswered questions and the final percentage score.

13. The completed test result is stored in PostgreSQL. Topic-wise performance and learning recommendation records are also updated.
14. The frontend displays the score, answer evaluations, correct answers, Bloom's Taxonomy levels, and explanations.
15. The user can access previous test records and download the assessment report in HTML format.

(b) Functional Requirements

The system provides the following functionalities:

- User Registration
- User Login and Authentication
- Password Hashing
- JWT-Based Authorization
- AI-Based MCQ Generation
- Subject and Topic Selection
- Difficulty-Level Selection
- Custom Number of Questions
- Test Duration Configuration
- Timed Online Examination
- Answer Selection and Tracking
- Automatic Test Submission on Timeout
- Manual Test Submission
- Automatic Answer Evaluation
- Score and Percentage Calculation
- Correct, Incorrect, and Unanswered Question Counts
- Bloom's Taxonomy Classification
- AI-Generated Answer Explanations
- Test History Management
- Topic-Wise Performance Tracking
- Rule-Based Learning Recommendations

- Assessment Report Download in HTML Format
- User Logout

(c) Non-functional Requirements

- **Usability:** The interface should be simple and easy to navigate.
- **Performance:** The application should process requests and display results efficiently, subject to external AI API response times.
- **Security:** Passwords should be stored as hashes, and protected API endpoints should require valid authentication.
- **Reliability:** The application should handle invalid requests and AI API errors without unnecessarily terminating the application.
- **Data Integrity:** User records, test records, and performance data should be maintained consistently in the database.
- **Privacy:** User credentials and assessment records should be protected against unauthorized access.
- **Maintainability:** The frontend, backend, database models, and AI generation logic should remain modular.
- **Scalability:** The architecture should support future enhancements and additional users with appropriate infrastructure changes.
- **Compatibility:** The web application should work with modern browsers.
- **Extensibility:** The system should permit future integration of additional AI models, analytics, and assessment features.
- **Content Quality:** AI-generated questions should be validated and reviewed for correctness and relevance where necessary.

6. Technical Requirements

Software Requirements

- **Operating System:** Windows 10/11 or Linux
- **Frontend Framework:** React.js
- **Frontend Build Tool:** Vite

- **Programming Languages:** Python and JavaScript
- **Backend Framework:** FastAPI
- **Database:** PostgreSQL
- **Object-Relational Mapper (ORM):** SQLAlchemy
- **AI Service:** Google Gemini API
- **AI Integration Library:** google.generativeai Python SDK
- **Authentication:** JSON Web Tokens (JWT)
- **JWT Library:** PyJWT
- **Password Hashing:** pwdlib with Argon2
- **Frontend Routing:** React Router (react-router-dom)
- **HTTP Client:** Axios
- **API Documentation and Testing:** FastAPI Swagger UI and Postman
- **Version Control:** Git and GitHub
- **Development Environment:** Visual Studio Code
- **Containerization:** Docker and Docker Compose
- **Browser:** Google Chrome or Microsoft Edge

Hardware Requirements

- A computer or laptop capable of running the development environment.
- Minimum 8 GB RAM recommended for local development.
- Sufficient storage for application code, dependencies, and database files.
- A stable internet connection for accessing the Google Gemini API.

Note: The hardware specifications are recommendations and may vary according to the deployment environment.

Database Requirements

The system uses PostgreSQL for persistent storage. SQLAlchemy manages database interactions through application models.

The principal database tables are:

1. **users:** Stores user identification details, email addresses, password hashes, and account status.
2. **tests:** Stores generated test sessions, user associations, selected parameters, test status, scores, and completion details.
3. **topic_performance:** Maintains aggregated topic-wise information, including attempted questions, correct answers, and accuracy.
4. **recommendation_history:** Stores learning recommendations generated according to assessment performance.

The actual question data, including correct answers and explanations, is temporarily maintained in server-side memory during an active test. The completed assessment results and related performance information are stored in PostgreSQL.

7. Expected Outcomes

The proposed system is designed to:

- Generate customized MCQs dynamically using Google Gemini.
- Reduce reliance on manually maintained question banks.
- Provide assessments based on the learner's selected subject, topic, and difficulty.
- Conduct timed examinations with automatic submission.
- Evaluate answers and calculate scores automatically.
- Display correct answers and detailed explanations after submission.
- Classify questions according to Bloom's Taxonomy.
- Maintain a history of completed tests.
- Track topic-wise accuracy and performance over time.
- Provide score-based learning recommendations.
- Reduce the manual effort involved in preparing practice question papers.
- Support academic preparation, competitive examinations, and placement interview practice.
- Demonstrate the practical integration of AI services with a modern full-stack web application.

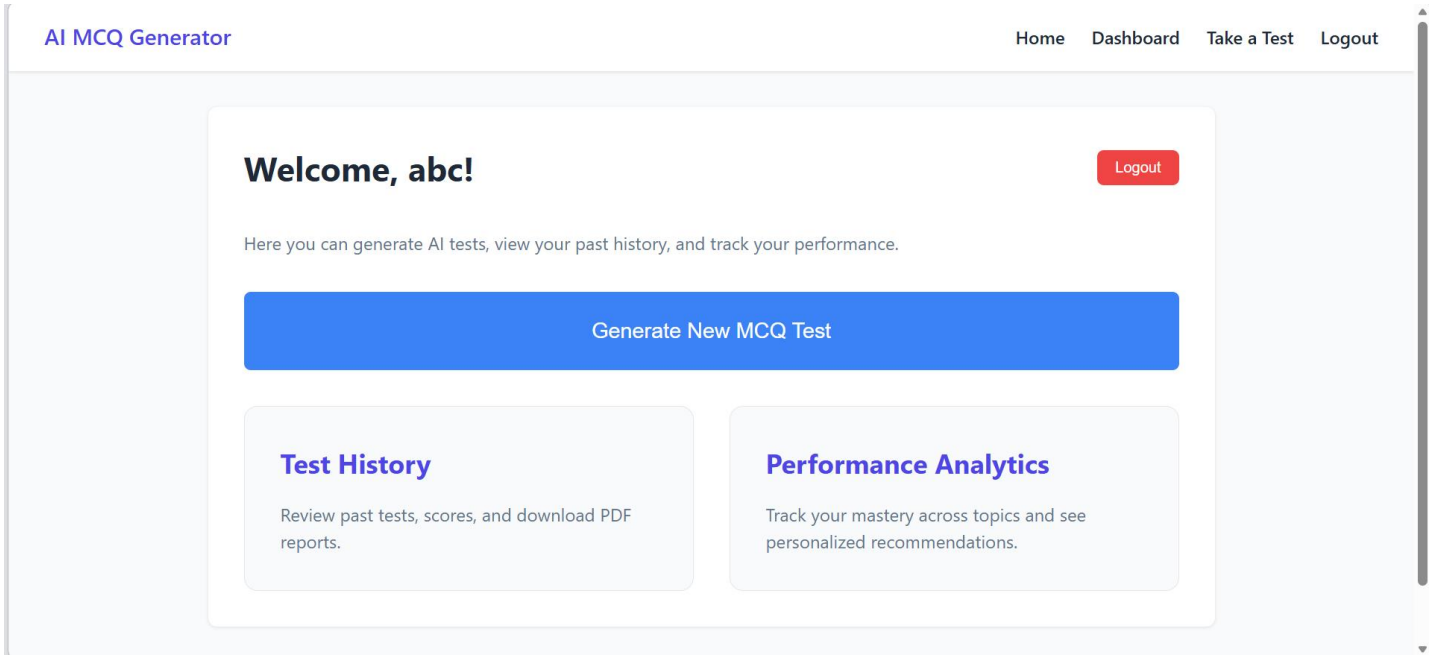
8. Conclusion

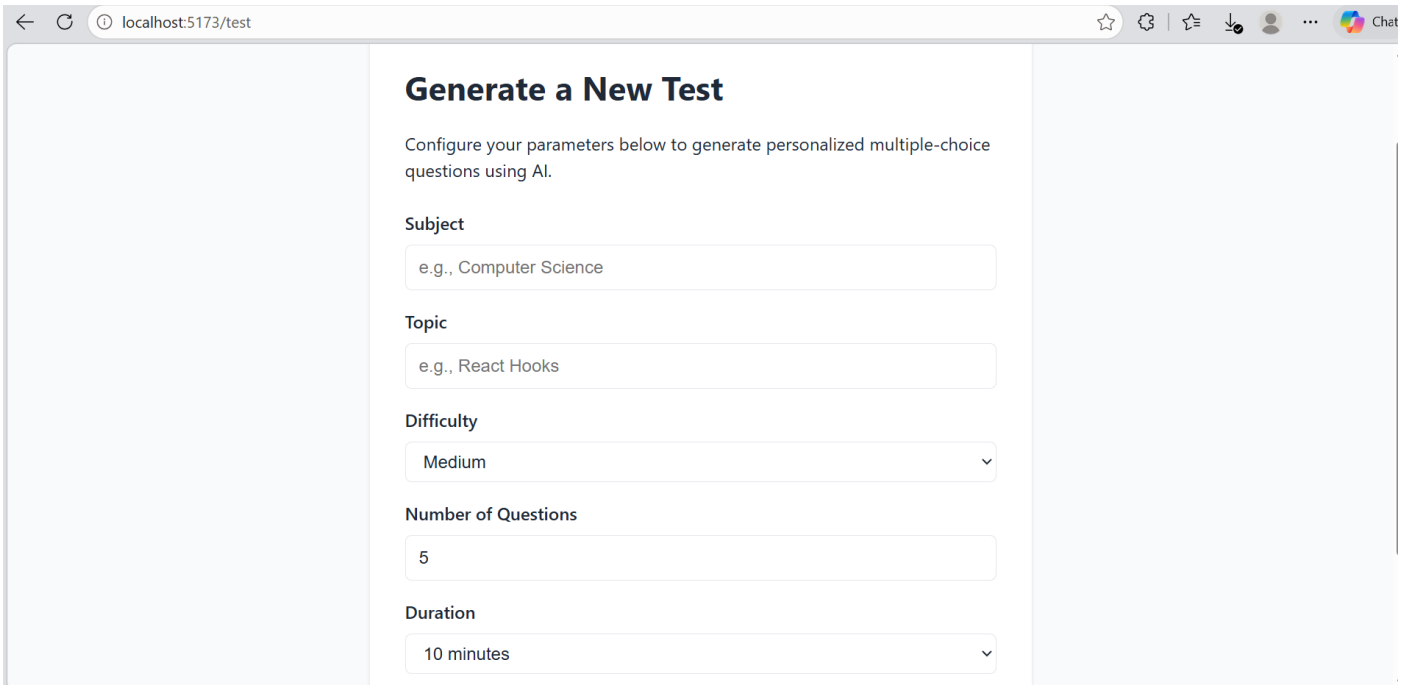
The AI-Powered Personalized MCQ Generator and Assessment Platform combines Artificial Intelligence with modern web technologies to provide a flexible and interactive online assessment system. The application uses React.js for the frontend, FastAPI for backend services, PostgreSQL for persistent data storage, and Google Gemini for dynamic MCQ generation.

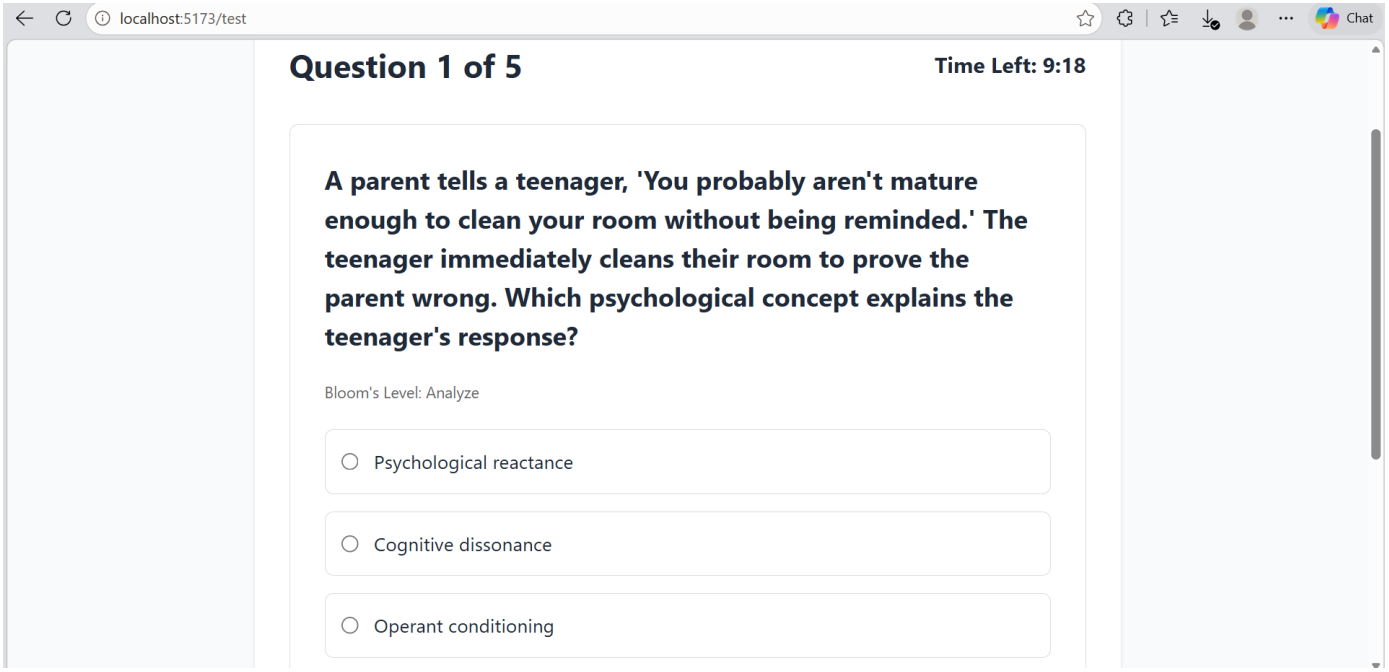
The system supports user authentication, customized test creation, timed examinations, automatic evaluation, answer explanations, test history, topic-wise performance tracking, and score-based learning recommendations. The inclusion of Bloom's Taxonomy classification adds a cognitive learning dimension to the generated assessments.

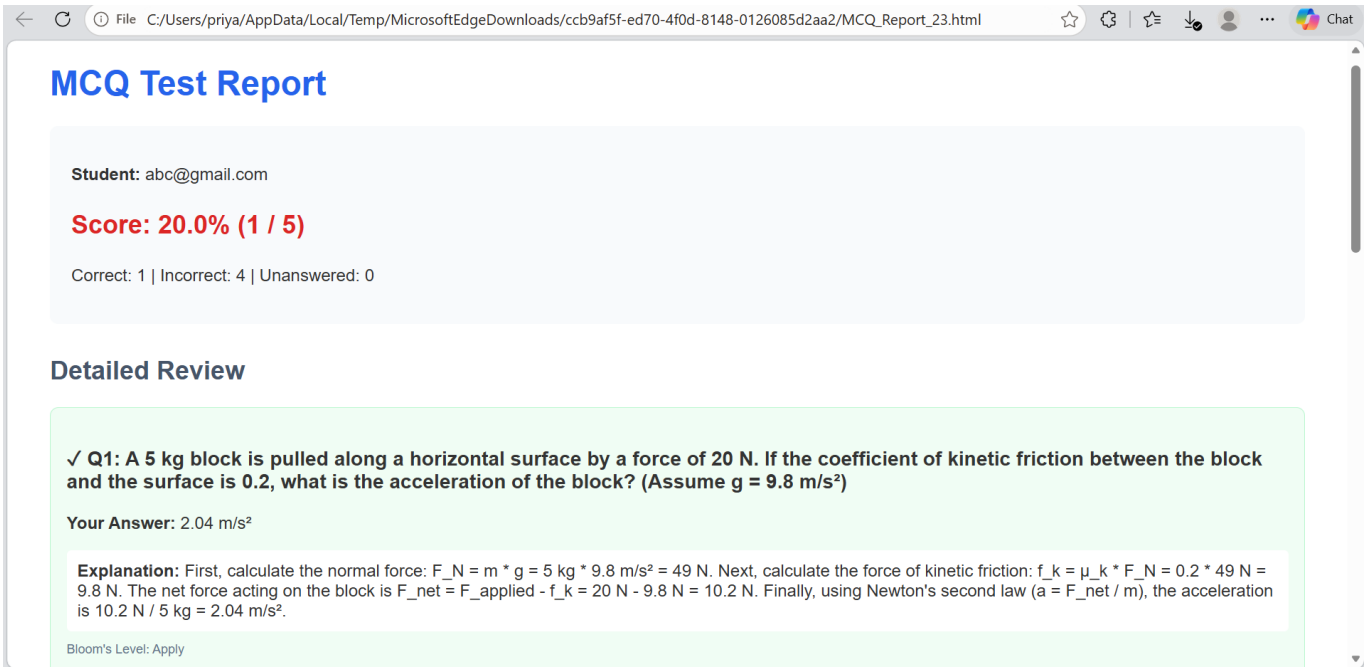
By integrating an external Large Language Model with a structured backend and relational database, the project demonstrates how AI can be applied to personalized learning and digital assessment. The platform provides a foundation for future improvements such as enhanced performance dashboards, robust PDF report generation, more advanced recommendation techniques, and integration with educational institutions.

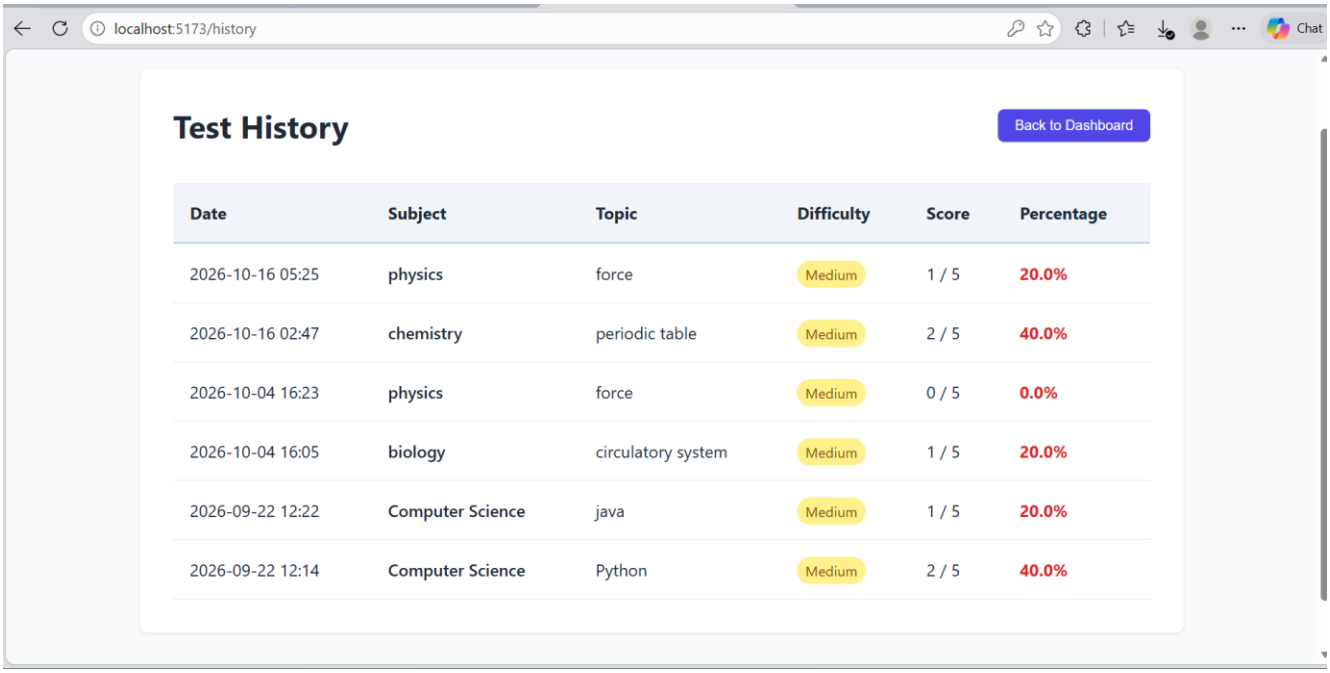
REPORT MINI PROJECT SCREENSHOTS

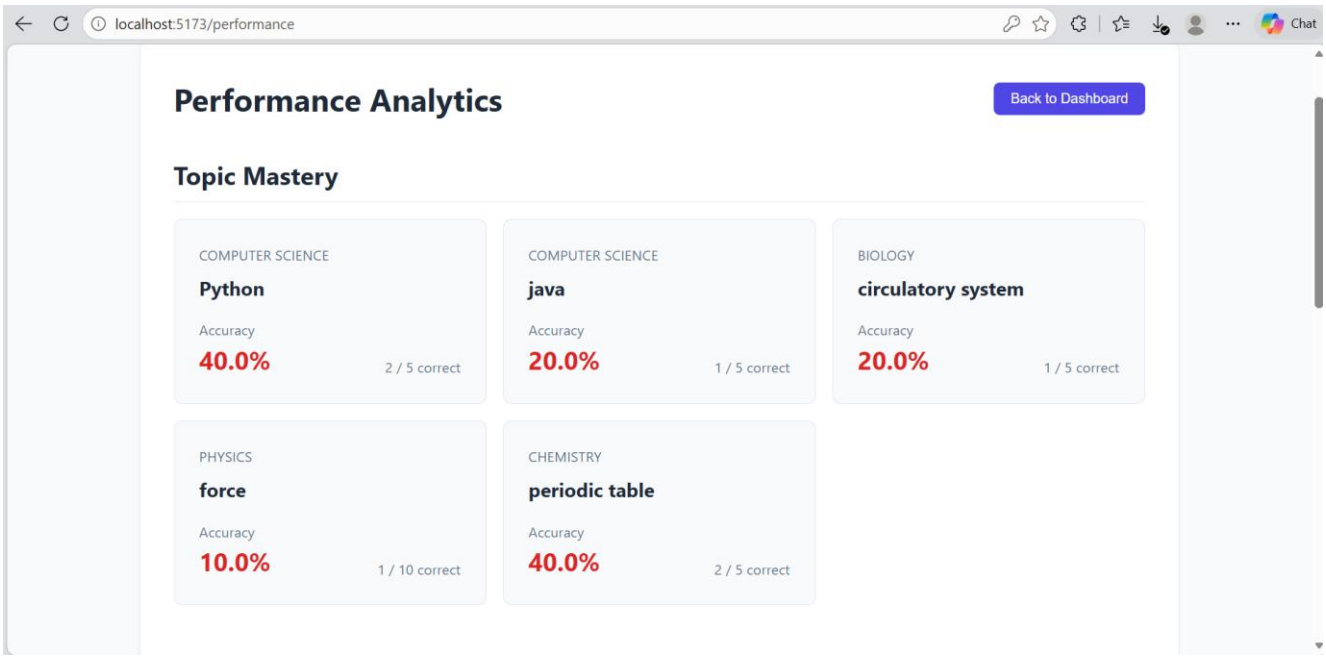



localhost:5173/test
Chat
Generate a New Test
Configure your parameters below to generate personalized multiple-choice questions using AI.
Subject
e.g., Computer Science
Topic
e.g., React Hooks
Difficulty
Medium
Number of Questions
5
Duration
10 minutes


localhost:5173/test
Chat
Question 1 of 5
Time Left: 9:18
A parent tells a teenager, 'You probably aren't mature enough to clean your room without being reminded.' The teenager immediately cleans their room to prove the parent wrong. Which psychological concept explains the teenager's response?
Bloom's Level: Analyze
Psychological reactance
Cognitive dissonance
Operant conditioning


File C:/Users/priya/AppData/Local/Temp/MicrosoftEdgeDownloads/ccb9af5f-ed70-4f0d-8148-0126085d2aa2/MCQ_Report_23.html
Chat
MCQ Test Report
Student: abc@gmail.com
Score: 20.0% (1 / 5)
Correct: 1 | Incorrect: 4 | Unanswered: 0
Detailed Review
✓ Q1: A 5 kg block is pulled along a horizontal surface by a force of 20 N. If the coefficient of kinetic friction between the block and the surface is 0.2, what is the acceleration of the block? (Assume g = 9.8 m/s²)
Your Answer: 2.04 m/s²
Explanation: First, calculate the normal force: F_N = m * g = 5 kg * 9.8 m/s² = 49 N. Next, calculate the force of kinetic friction: f_k = μ_k * F_N = 0.2 * 49 N = 9.8 N. The net force acting on the block is F_net = F_applied - f_k = 20 N - 9.8 N = 10.2 N. Finally, using Newton's second law (a = F_net / m), the acceleration is 10.2 N / 5 kg = 2.04 m/s².
Bloom's Level: Apply


localhost:5173/history
Chat
Test History
Back to Dashboard
Date
Subject
Topic
Difficulty
Score
Percentage
2026-10-16 05:25
physics
force
Medium
1 / 5
20.0%
2026-10-16 02:47
chemistry
periodic table
Medium
2 / 5
40.0%
2026-10-04 16:23
physics
force
Medium
0 / 5
0.0%
2026-10-04 16:05
biology
circulatory system
Medium
1 / 5
20.0%
2026-09-22 12:22
Computer Science
java
Medium
1 / 5
20.0%
2026-09-22 12:14
Computer Science
Python
Medium
2 / 5
40.0%


localhost:5173/performance
Chat
Performance Analytics
Back to Dashboard
Topic Mastery
COMPUTER SCIENCE
Python
Accuracy
40.0%
2 / 5 correct
COMPUTER SCIENCE
java
Accuracy
20.0%
1 / 5 correct
BIOLOGY
circulatory system
Accuracy
20.0%
1 / 5 correct
PHYSICS
force
Accuracy
10.0%
1 / 10 correct
CHEMISTRY
periodic table
Accuracy
40.0%
2 / 5 correct